

Final project

Probabilistic Inference in Reinforcement Learning

Marco Cartago SM3800130, Marco Chiorri SM3800149

Our Work

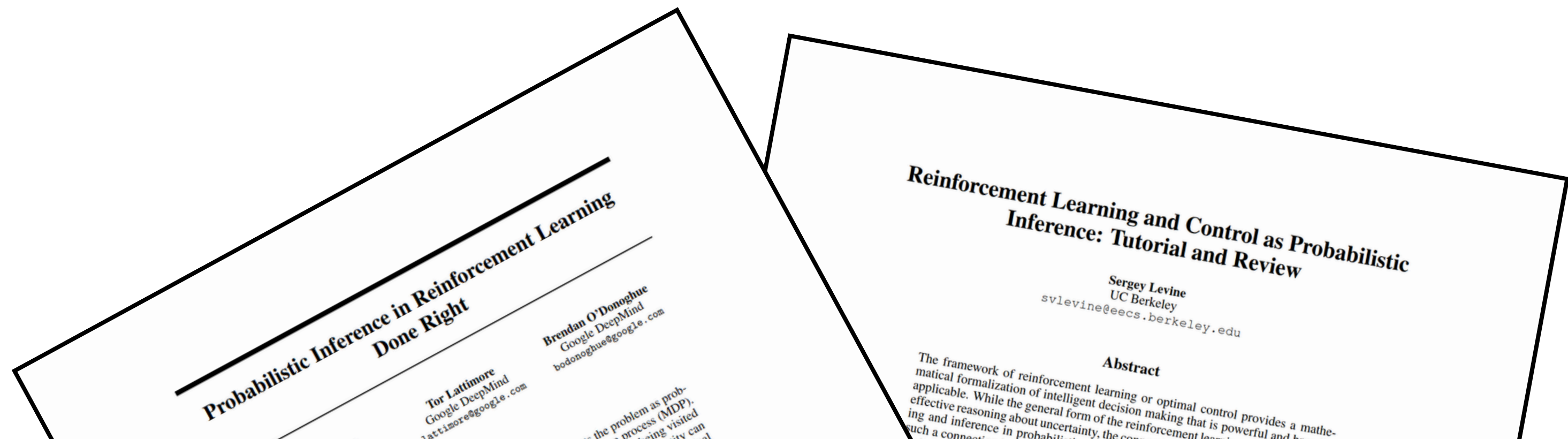
We were interested in the approach proposed by the **reinforcement learning as inference** framework.

We focused on two main works:

- [1] **Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review**, by *Sergey Levine*
- [2] **Probabilistic Inference in Reinforcement Learning Done Right**, by *Jean Tarbouriech, Tor Lattimore, Brendan O'Donoghue*,

In this presentation will first introduce from a theoretical point of view the techniques presented by **Levine** in [1] followed by their refinement in [2] by **O'Donoghue**.

In the second part of the presentation we will present the results simulations in a simple gridworld environment to assess the learning capabilities of the different learning strategies with Q-learning as a baseline.



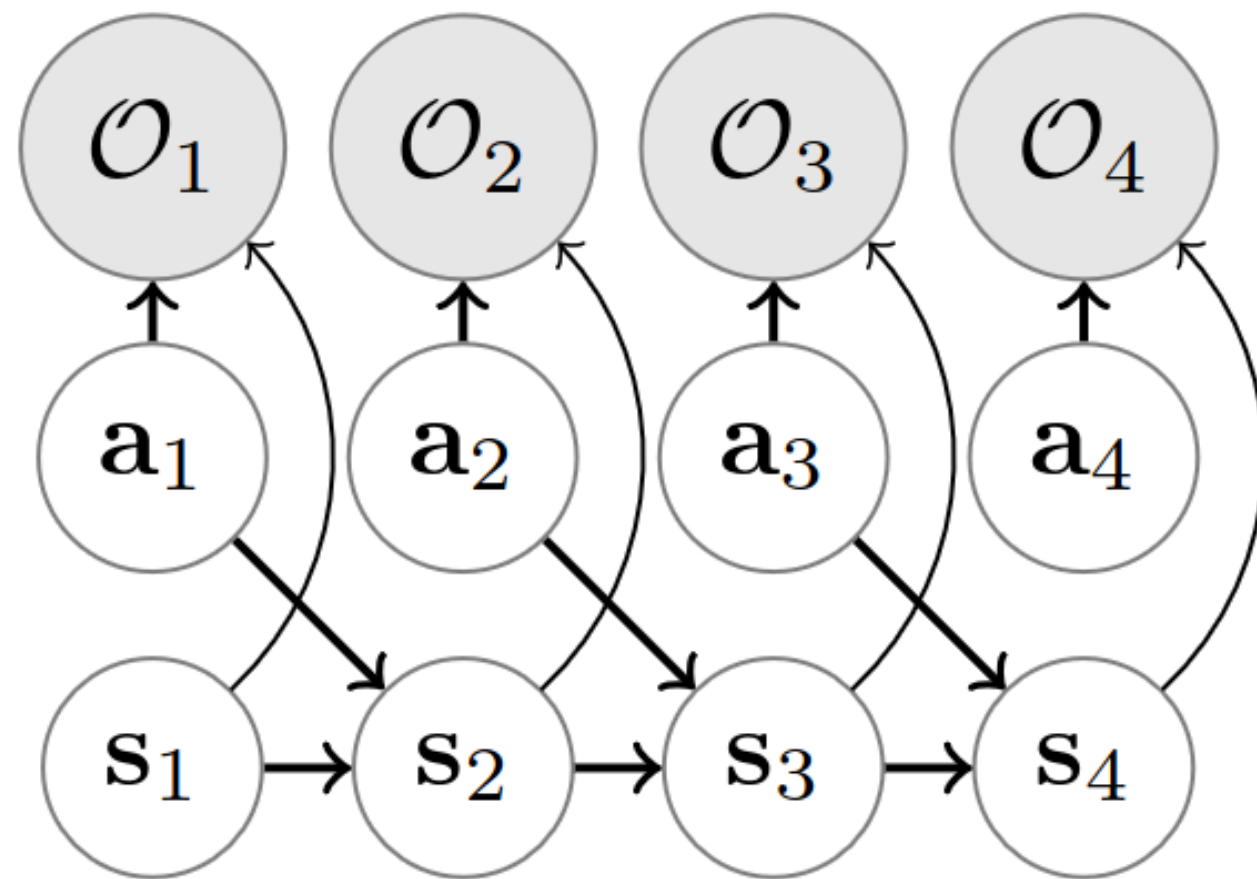
RL as an inference problem

The key idea to treat RL as an inference problem is to add to the structure of the MDP an **optimality variable**. Variable that represents the probability of said state to be visited under the optimal policy.

This variable $\mathcal{O}_t$ (or Γ^*) is then defined as having the density:

$$p(\mathcal{O}_t = 1 | s_t, a_t) = \exp(r(s_t, \mathbf{a}_t))$$

The whole problem can then be expressed by the following graphical model:



The remarkable thing is that by solving the model via message passing we obtain naturally a sort of smoothed bellman update from which one seems to be able to define values and Q-values

$$\beta(s_t, a_t) = p(\mathcal{O}_{t:T} | s_t, a_t)$$

$$\beta(s_t) = p(\mathcal{O}_{t:T} | s_t)$$

$$Q(s_t, a_t) = \log \beta(s_t, a_t)$$

$$V(s_t) = \log \beta(s_t)$$

$$V(s_t) = \log \int_{\mathcal{A}} \exp(Q(s_t, a_t)) da_t$$

The optimal policy is then expressed as: $p(\mathbf{a}_t | s_t, \mathcal{O}_t = 1)$

Extension to stochastic environments

There are some problems with this approach, because if we are considering a deterministic environment this works perfectly, but if we happen to work in a stochastic environment, this probability is going to give us a lot of problems.

$$p(s_{t+1} | s_t, a_t, \mathcal{O}_{1:T})$$

We are implicitly saying that the probability of going into another state depends also on the optimality variables. So the probability can change based on different values. This is like allowing the agent to interfere with the dynamics of the environment. The solution to the problem on the other hand is quite easy, we just fix it to be exactly the same, so:

$$p(s_{t+1} | s_t, a_t, \mathcal{O}_{1:T}) = p(s_{t+1} | s_t, a_t)$$

Links to Maximum Entropy RL

It turns out that following this kind of approach produces a formulation of another approach: **maximum entropy RL**. In which the aim of the agent becomes the sum of the rewards and of the entropy, not only the rewards anymore.

$$-D_{KL}(\hat{p}(\tau)||p(\tau)) = \sum_{t=1}^T \mathbb{E}_{(s_t, a_t) \sim \hat{p}(s_t, a_t)} [r(s_t, a_t)] + \mathbb{E}_{(s_t) \sim \hat{p}(s_t)} [\mathcal{H}(\pi(a_t|s_t))]$$



$$\begin{aligned} p(\tau|\mathcal{O}_{1:T}) &\propto p(\tau, \mathcal{O}_{1:T}) = p(s_1) \prod_{t=1}^T p(\mathcal{O}_t|s_t, a_t) p(s_{t+1}|s_t, a_t) \\ &= p(s_1) \prod_{t=1}^T \exp(r(s_t, a_t)) p(s_{t+1}|s_t, a_t) \\ &= \left[p(s_1) \prod_{t=1}^T p(s_{t+1}|s_t, a_t) \right] \exp \left(\sum_{t=1}^T r(s_t, a_t) \right) \end{aligned}$$

Soft Q-learning

From these assumptions, starting from the variational approximation of the policy, one can derive an update scheme valid for gradient based methods but that can be used to derive a “soft-update” equation for the method (in our case the integral is replaced with a summation)

$$\phi \leftarrow \phi - \alpha E \left[\frac{dQ_\phi}{d\phi}(\mathbf{s}_t, \mathbf{a}_t) \left(Q_\phi(\mathbf{s}_t, \mathbf{a}_t) - \left(r(\mathbf{s}_t, \mathbf{a}_t) + \log \int_{\mathcal{A}} \exp(Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})) d\mathbf{a}_{t+1} \right) \right) \right]$$

Which gives the following tabular method:

$$Q(\mathbf{s}_t, \mathbf{a}_t) \leftarrow Q(\mathbf{s}_t, \mathbf{a}_t) - \alpha \left[Q(\mathbf{s}_t, \mathbf{a}_t) - \left(r(\mathbf{s}_t, \mathbf{a}_t) + \log \int_{\mathcal{A}} \exp(Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})) d\mathbf{a}_{t+1} \right) \right]$$

Although this is the true formulation of the Soft Q learning, we implemented a different version with the temperature by the implementation of the rewards.

(Reinforcement Learning with Deep Energy-Based Policies)

$$\tau \cdot \log \int_{\mathcal{A}} \exp \left(\frac{Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})}{\tau} \right) d\mathbf{a}_{t+1}$$

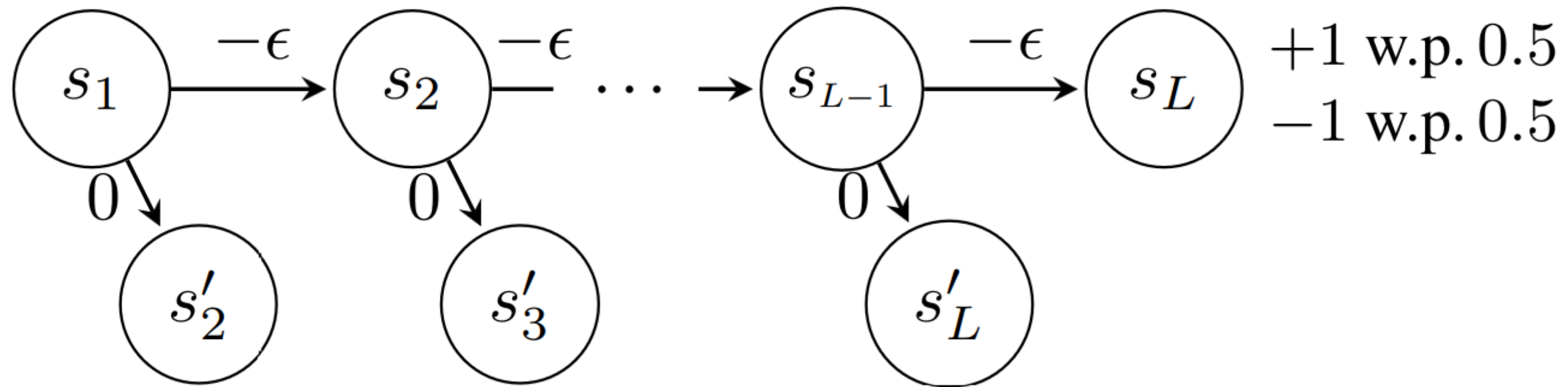

The resulting policy then becomes: $\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp \left(\frac{Q(\mathbf{s}_t, \mathbf{a}_t) - V(\mathbf{s}_t)}{\tau} \right)$

Where τ has the effectively the role of tuning the strength of the entropy term.

A simple (hard) problem

Although **Soft-Q-learning** is able to learn effectively in many circumstances it can be show how, even in a simple enviroment, the learning effectiveness of the agent is compromised. In this kind of scenario Soft-Q-learning learns with an episode count of $\Omega(2^L)$

Assuming $\epsilon L \ll 1$



Soft-Q-Learning takes an exponential amount of episodes to reach the optimal policy because is not completely taking advantage of the epistemologic uncertainty of the model, is just doing a frequentist approach making no distinction from when we have explored that way a lot of times (having small variance) or being uncertain (having big variance).

Variational **A**pproximation of the posterior Probability of **O**ptimality in **R**einforcement learning

Reformulation

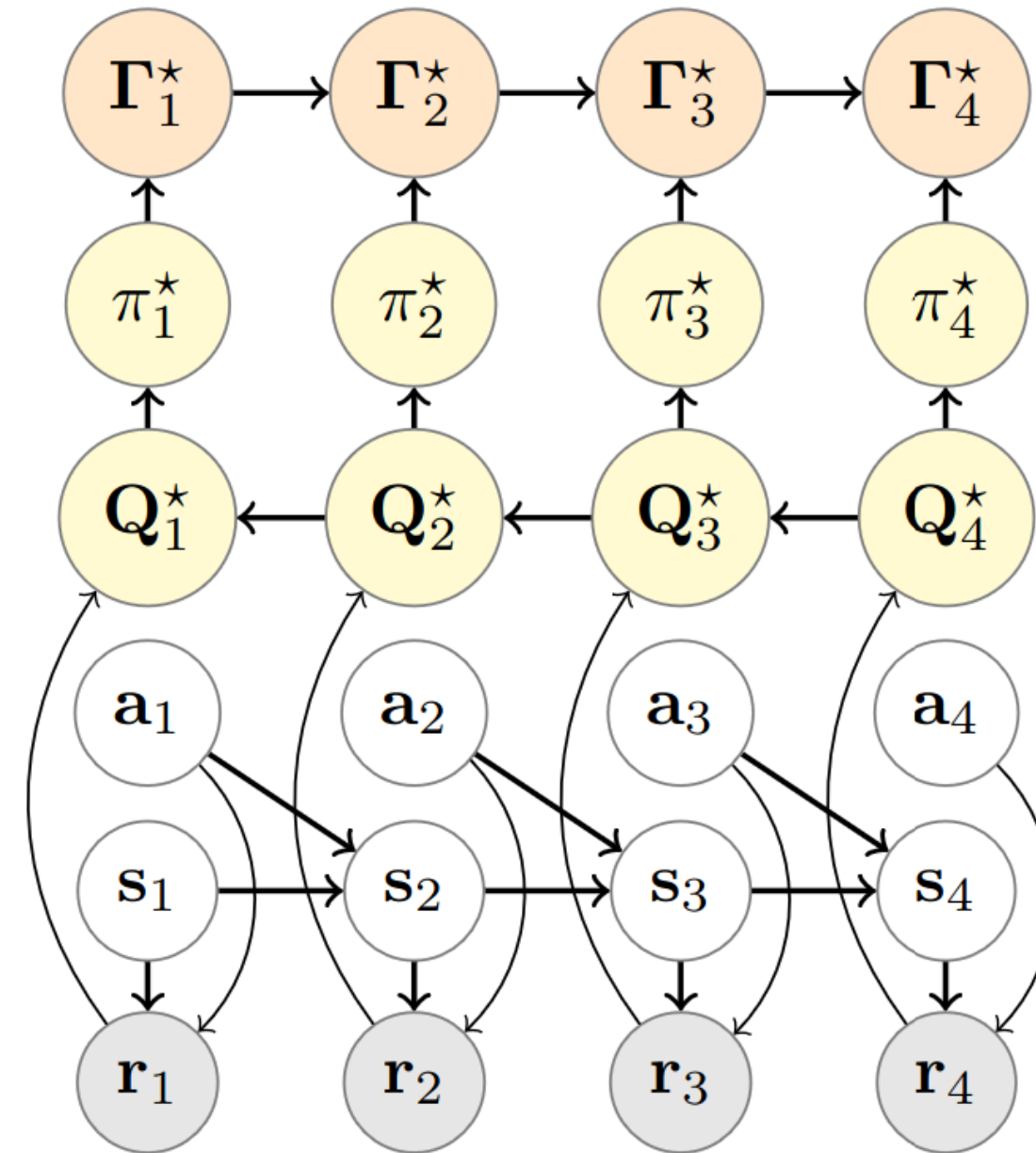
We define in a new way the optimal variables, based not only on the state and action in that moment but in a recursive fashion. Starting from the newly defined variables one can then proceed, through the optimal policy to define the other optimality variables such as $\mathbf{Q}^*$ and the optimal policy. This can then be expressed by the means of a graphical model:

$$\Gamma_1^*(s) := \{s_1 = s\},$$

$$\Gamma_{l+1}^*(s') := \bigcup_{s \in S_l, a \in A} \Gamma_l^*(s, a) \cap \{s_{l+1} = s'\},$$

$$\Gamma_1^*(s, a) := \Gamma_1^*(s) \cap \{\pi_1^*(s) = a\},$$

$$\Gamma_{l+1}^*(s', a') := \Gamma_{l+1}^*(s') \cap \{\pi_{l+1}^*(s') = a'\}.$$



Lemmas (1)

Before starting, a key definition is the one of the set of occupancy measures: which will be the target space for us to reason about optimality. Because, as stated in **Lemma 1**, the optimal policy can be defined in terms of its occupancy measures.

$$\Lambda(P) := \{\lambda \in \mathbb{R}_+^{L,S,A} : \sum_a \lambda_1(s, a) = \rho(s) \sum_a \lambda_{l+1}(s', a') = \sum_{s,a} P_l(s'|s, a) \lambda_l(s, a)\}$$

Lemma 1 For known P , we have that the probability of each pair state-action being visited under the optimal policy is part of the set of occupancy measures defined for the system:

$$\mathbb{P}_{\Gamma^*} := \{\mathbb{P}_\phi(\Gamma_l^*)\}_{l=1}^L \in \Lambda(P)$$

The way in which we can recover the optimal policy is than:

$$\pi_l(s, a) = \frac{\mathbb{P}_\phi(\Gamma_l^*(s, a))}{\sum_{a' \in \mathcal{A}} \mathbb{P}_\phi(\Gamma_l^*(s, a'))} = \mathbb{P}_\phi(\pi_l^*(s) = a | \Gamma_l^*(s))$$

Lemmas (2)

$$\mathbb{E}_{s \sim \rho} \mathbb{E}_{\phi} V_1^*(s) = \sum_{l,s,a} \mathbb{P}_{\phi}(\Gamma_l^*(s,a)) \mathbb{E}_{\phi}[r_l(s,a) \mid \Gamma_l^*(s,a)]$$

Lemma 2

We can express the expected optimal value from a certain policy through this summation, that relates the expected optimal value, to the occupancy measures.

Lemma 3

If we make the assumption that the reward is sub-gaussian we can bound the expected reward

$$\begin{aligned} \mathbb{E}_{\phi}[r_l(s,a) \mid \Gamma_l^*(s,a)] &\leq \mathbb{E}_{\phi} r_l(s,a) + \min_{\tau_l(s,a) > 0} \left(\frac{\sigma_l^2(s,a)}{2\tau_l(s,a)} - \tau_l(s,a) \log \mathbb{P}_{\phi}(\Gamma_l^*(s,a)) \right) \\ &= \mathbb{E}_{\phi} r_l(s,a) + \sigma_l(s,a) \sqrt{-2 \log \mathbb{P}_{\phi}(\Gamma_l^*(s,a))}. \end{aligned}$$

VAPOR objective

$$\mathcal{V}_{\phi}(\lambda, \tau) := \lambda^{\top} \left(\mathbb{E}_{\phi} r + \frac{\sigma^2}{2\tau} \right) + \mathcal{H}_{\tau}(\lambda),$$

$$\mathcal{V}_{\phi}(\lambda) := \min_{\tau \in \mathbb{R}_+^{L,S,A}} \mathcal{V}_{\phi}(\lambda, \tau) = \lambda^{\top} \left(\mathbb{E}_{\phi} r + \sigma \circ \sqrt{-2 \log \lambda} \right).$$

Finally, by combining Lemmas 2 and 3 we obtain a concave function in $\mathbf{P}\Gamma^*$ that upper bounds the value function objective.

Lemmas (3)

Lemma 4 By combining all of the lemmas we arrive at this sequence of inequalities, we are searching the maximum of VAPOR objective because it is the only part we can evaluate:

$$\mathbb{E}_{s \sim \rho} \mathbb{E}_{\phi} V_1^*(s) \leq \mathcal{V}_{\phi}(\mathbb{P}_{\Gamma^*}) \leq \max_{\lambda \in \Lambda(P)} \mathcal{V}_{\phi}(\lambda)$$

Lemma 5 Is the final piece that guarantees that the variational approximation we are performing “is close” to the true solution, saying that if we have a tighter upper bound of the previous formula then we have a more accurate estimation of the probability of the optimal variables

$$D_{KL_{\tau^*}}(\mathbb{P}_{\Gamma^*} \parallel \lambda^*) \leq \mathcal{V}_{\phi}(\lambda^*) - \mathbb{E}_{s \sim p} \mathbb{E}_{\phi} V_1^*(s)$$

VAPOR algorithm

The training loop of the tabular method VAPOR takes the following form:

Initialize ϕ belief state

—————
Initialized to be independent zero mean
gaussians with the same variance.

for $t \in 1:T$ **do**

Compute expectations wrt ϕ

$R \leftarrow$ Rewards $E^t r$

$P \leftarrow$ Transitions $E^t P$

$S \leftarrow$ Uncertainty measures σ

—————
We did not pose a Dirichlet prior on the
transitions, we calculated them analytically.

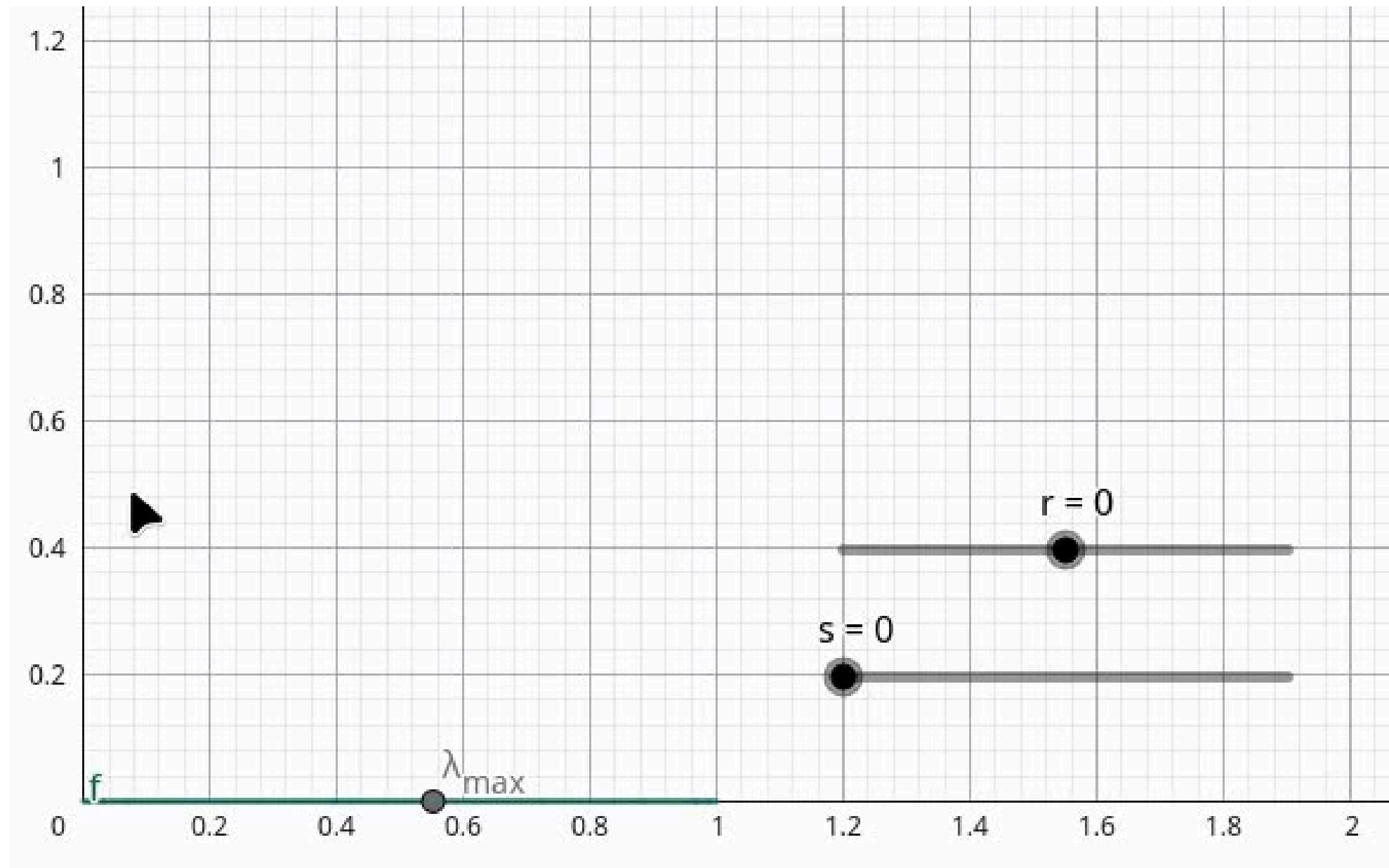
$\lambda \leftarrow \operatorname{argmax} (x) \langle x; R + S \circ \sqrt{-2 \log x} \rangle$
 subject to occupancy constraints (P)

—————
This is a Convex problem which admits a unique solution.
For solving we used **CVXPY**, in combination with the
CLARABEL solver with an absolute tolerance of $1e-5$.

Execute $\pi(s, a) \propto \lambda(s, a)$ for L steps

end

Visualization of the VAPOR objective



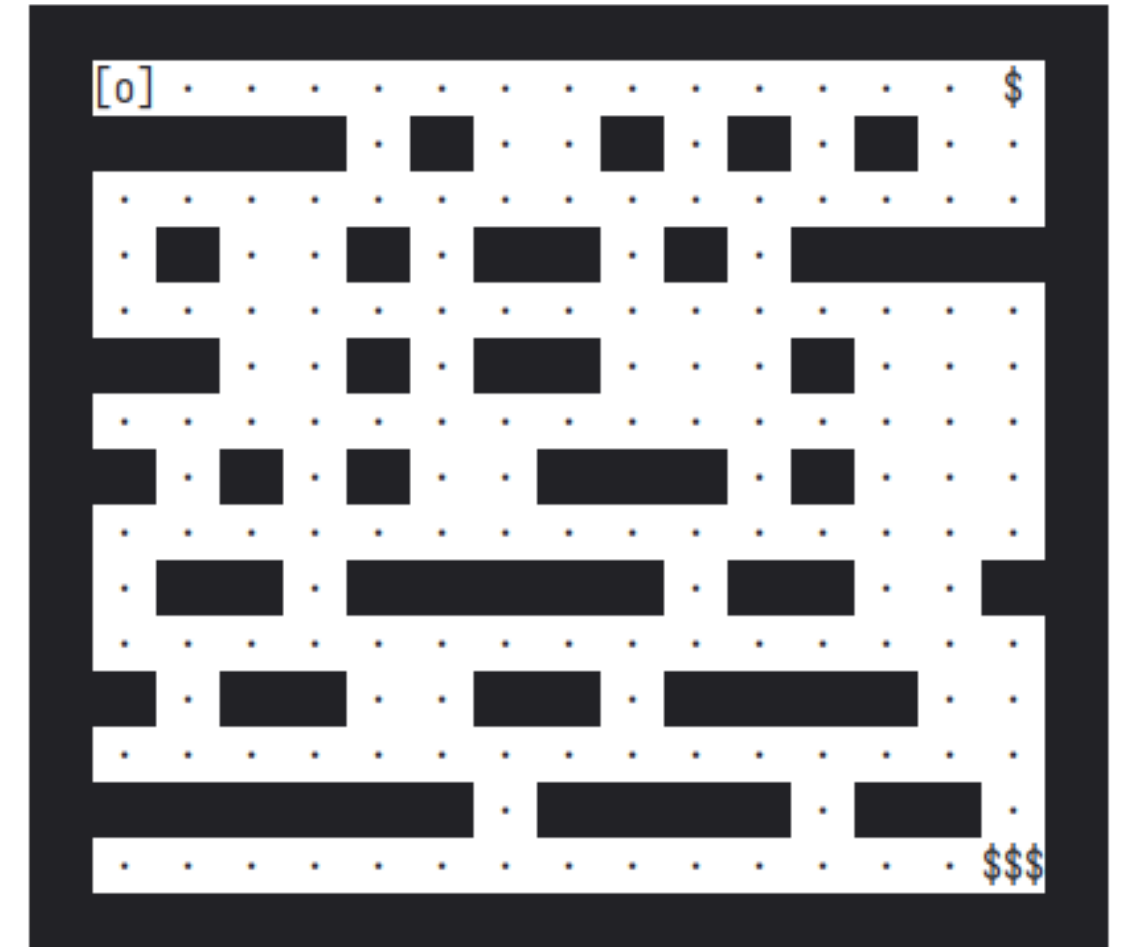
$$\mathcal{V}_{\phi}(\lambda) := \lambda^{\top} \left(\mathbb{E}_{\phi} r + \sigma \circ \sqrt{-2 \log \lambda} \right).$$

The enviroment

The environment is a gridworld of size $N \times N$, there are 2 possible goals: the easiest is just by going right to the end of the gridworld but it has a very little reward, instead the other is in the opposite corner and it takes more time to arrive but it is worthy, it gives a bigger reward.

For each step that is made in a direction of a goal (so either right or down) there is a little negative reward so the agent is motivated to reach the goal, but there is this negative reward only by going right or down. If it isn't able to reach one of the goals in a given time, it will receive a big negative reward.

We decided to keep the maximum number of steps at: $2 \cdot (\text{size} + 1)$



Notes on the environment

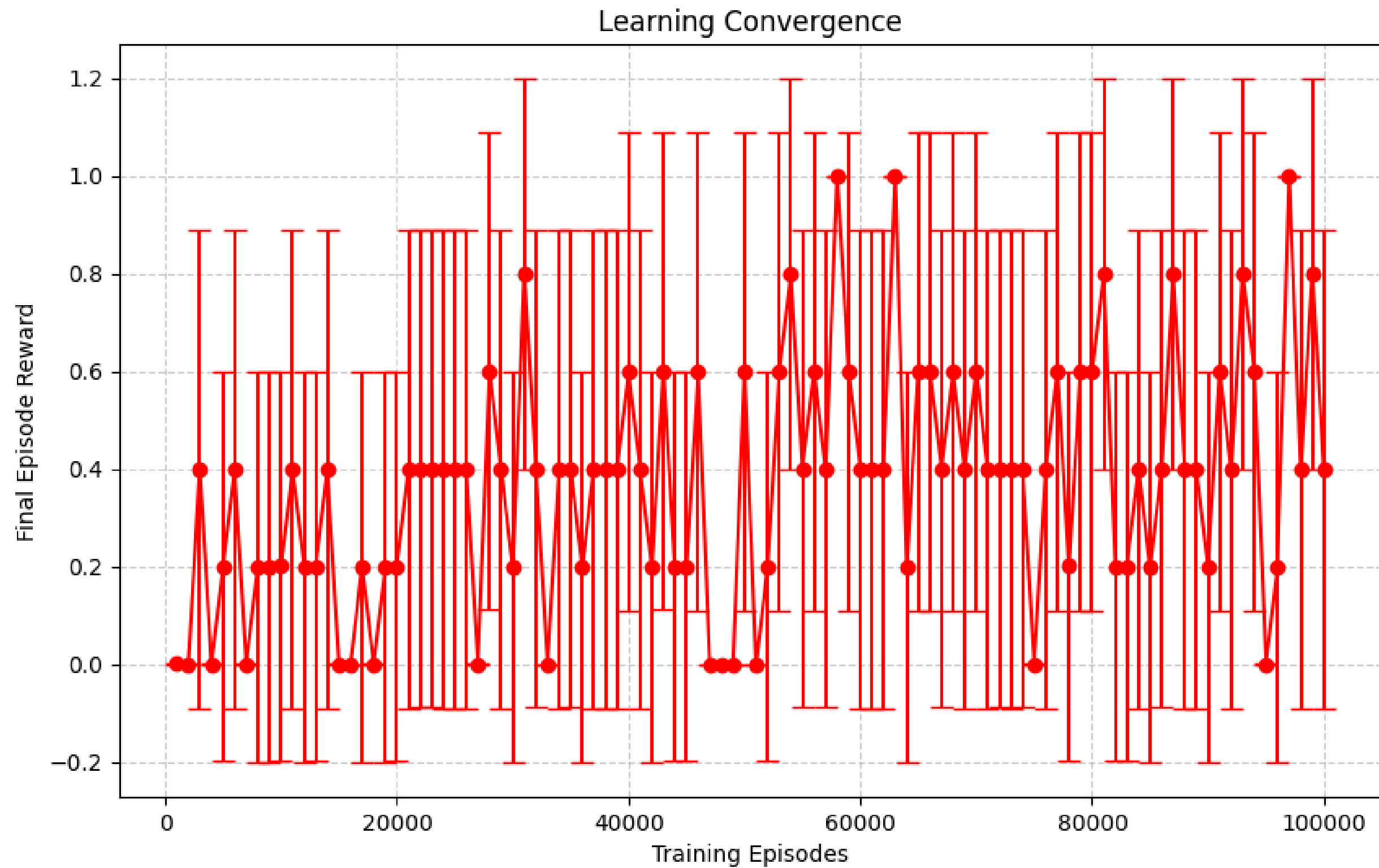
We know that it could take less time to reach the goal in most of the examples but to avoid the possibility of being too much wall we decided to adopt a more loose bound, but not too loose because the VAPOR implementation would require too many lambda variables, in fact we achieved a maximum size for the test of the environment of 12×12 , after that the process was killed for not having enough memory.

Q-learning

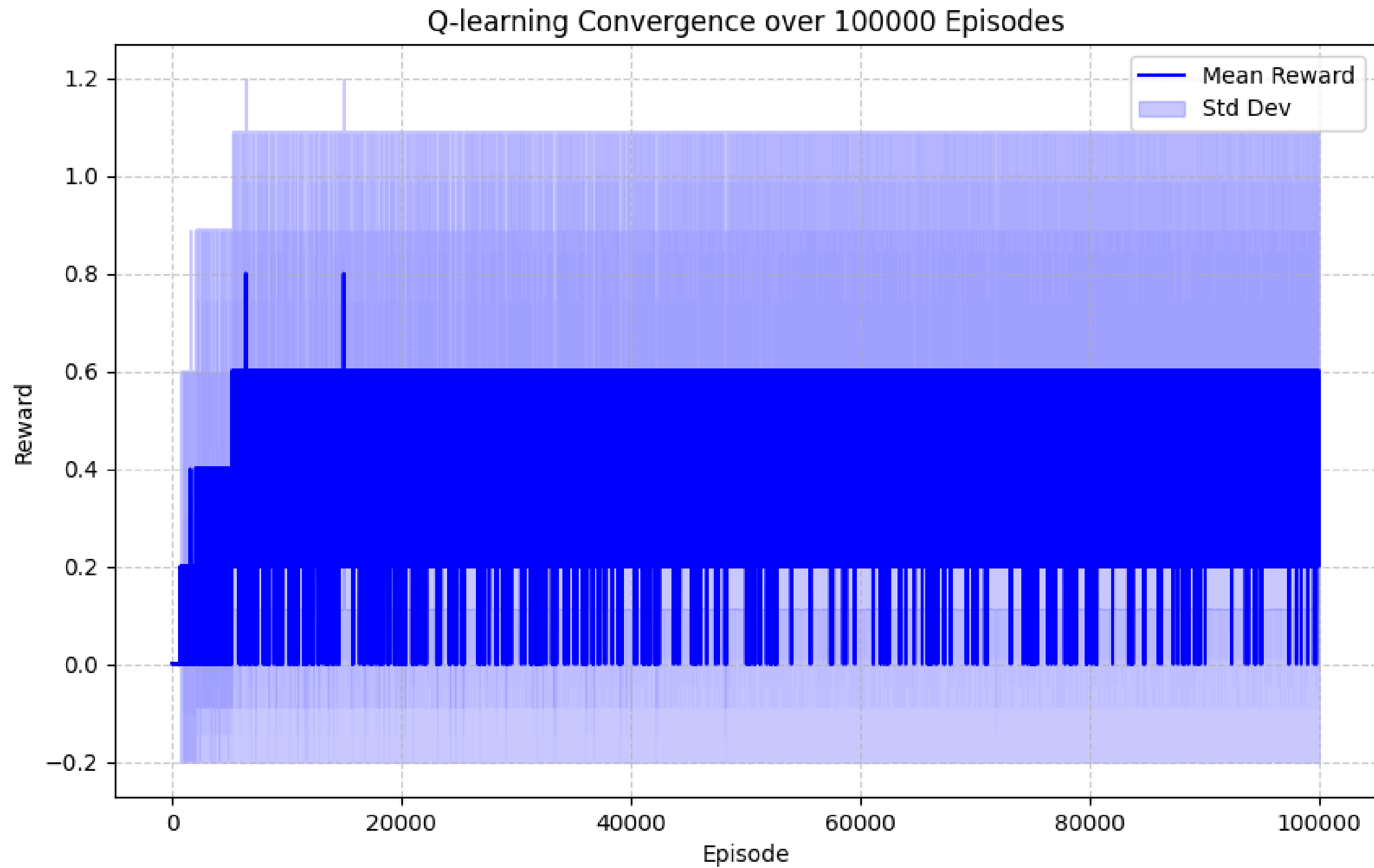
Results: Q-learning



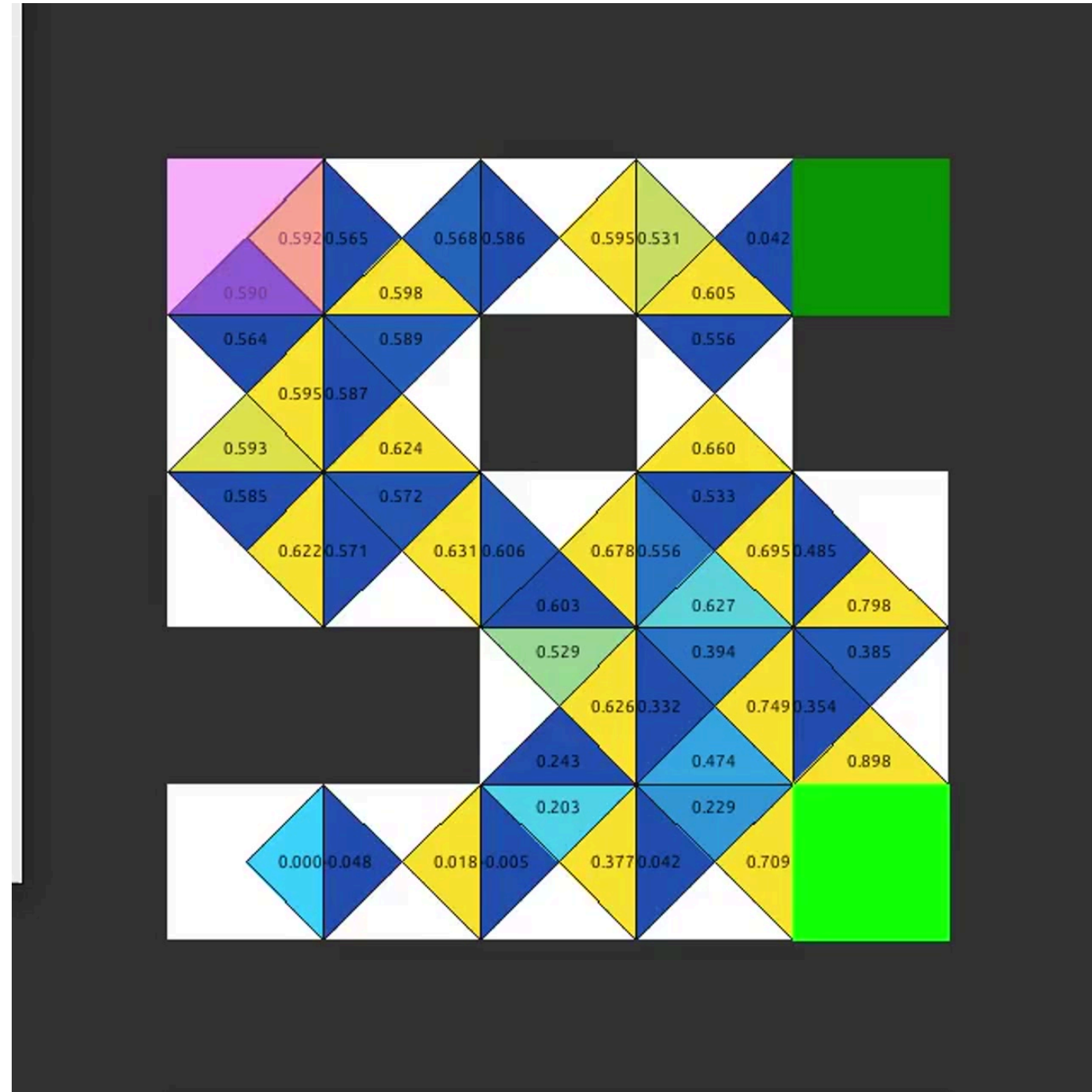
Results: Q-learning



Results: Q-learning

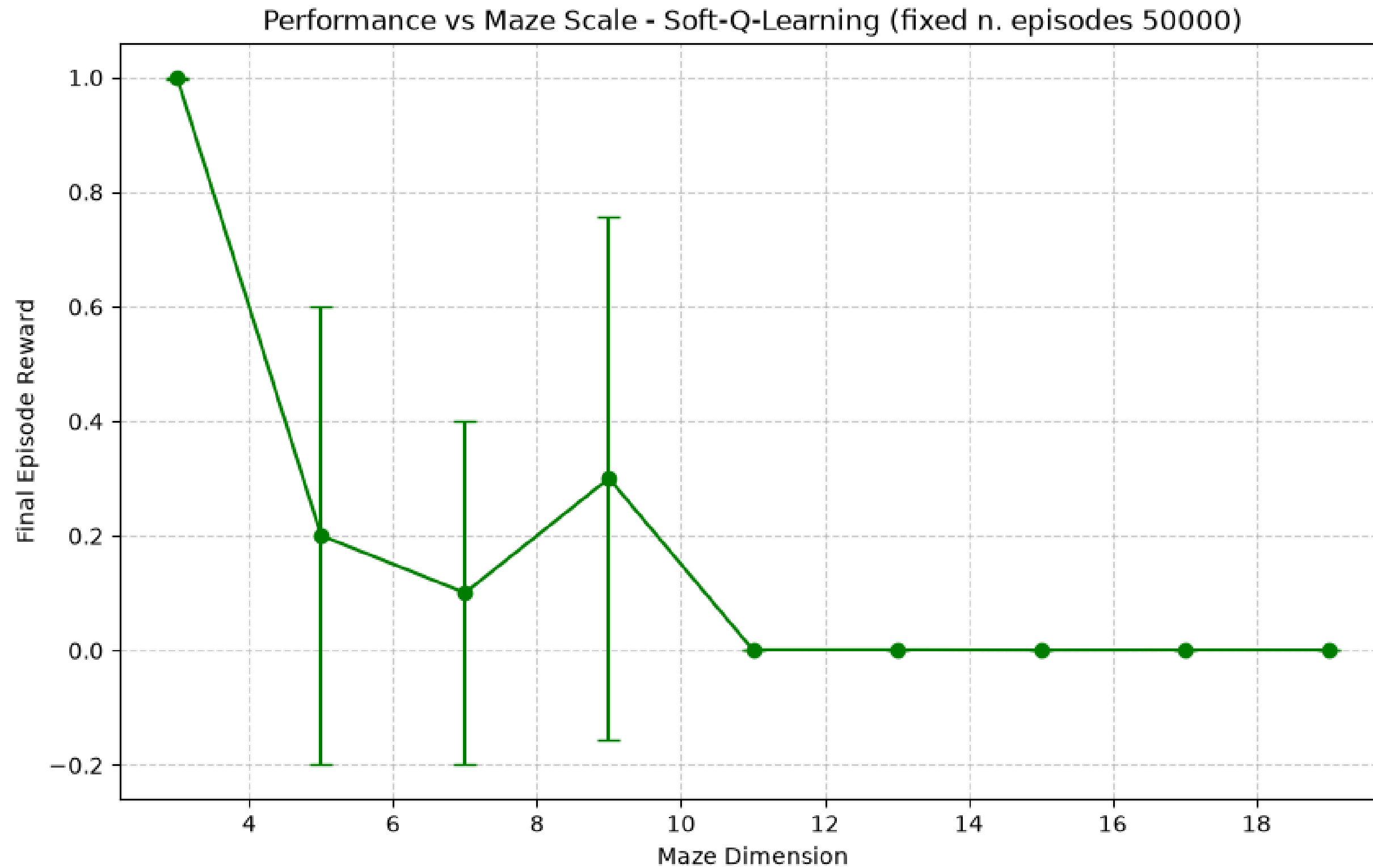


Results: Q-learning

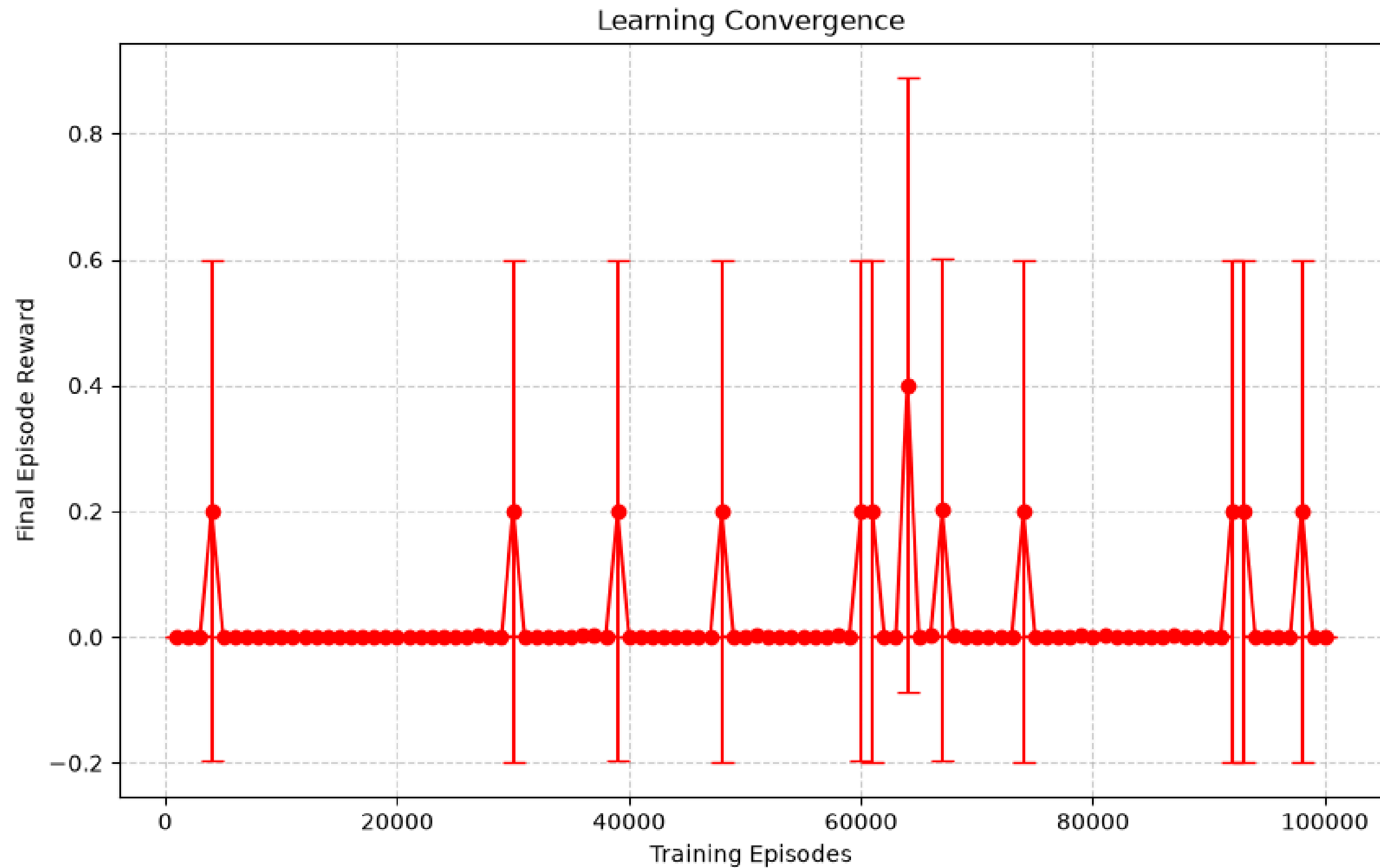


Soft Q-learning

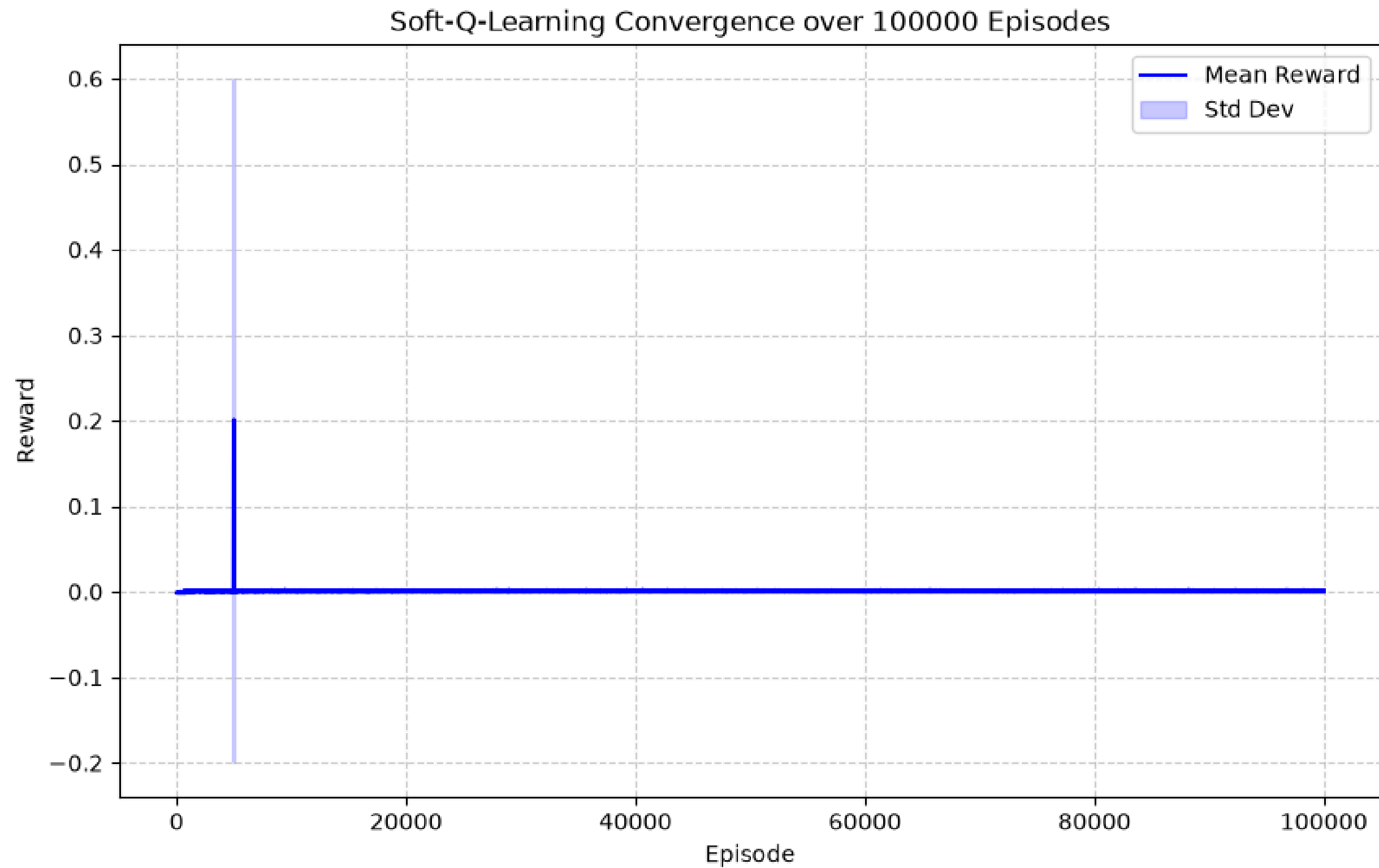
Results: Soft Q-learning



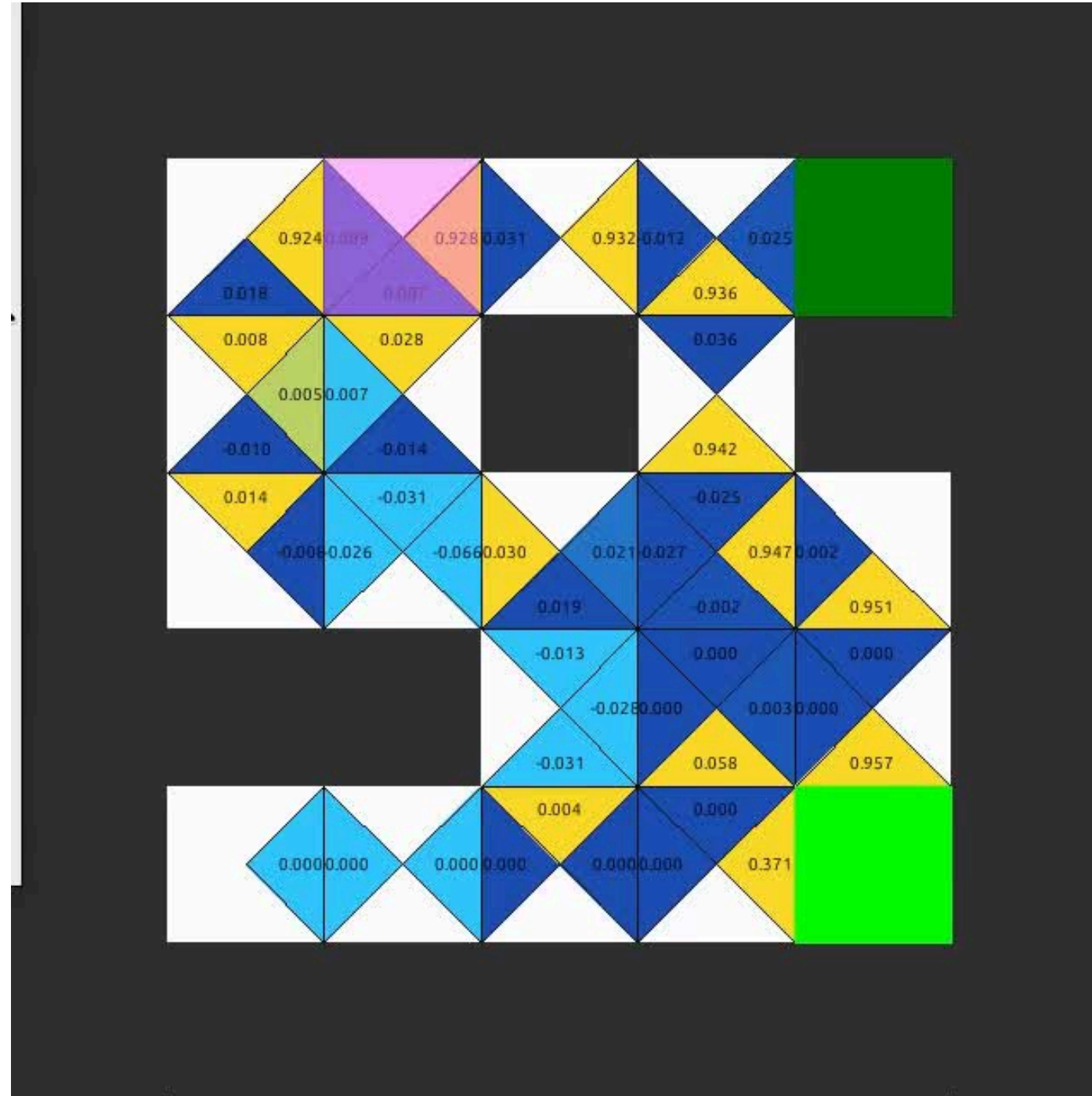
Results: Soft Q-learning



Results: Soft Q-learning

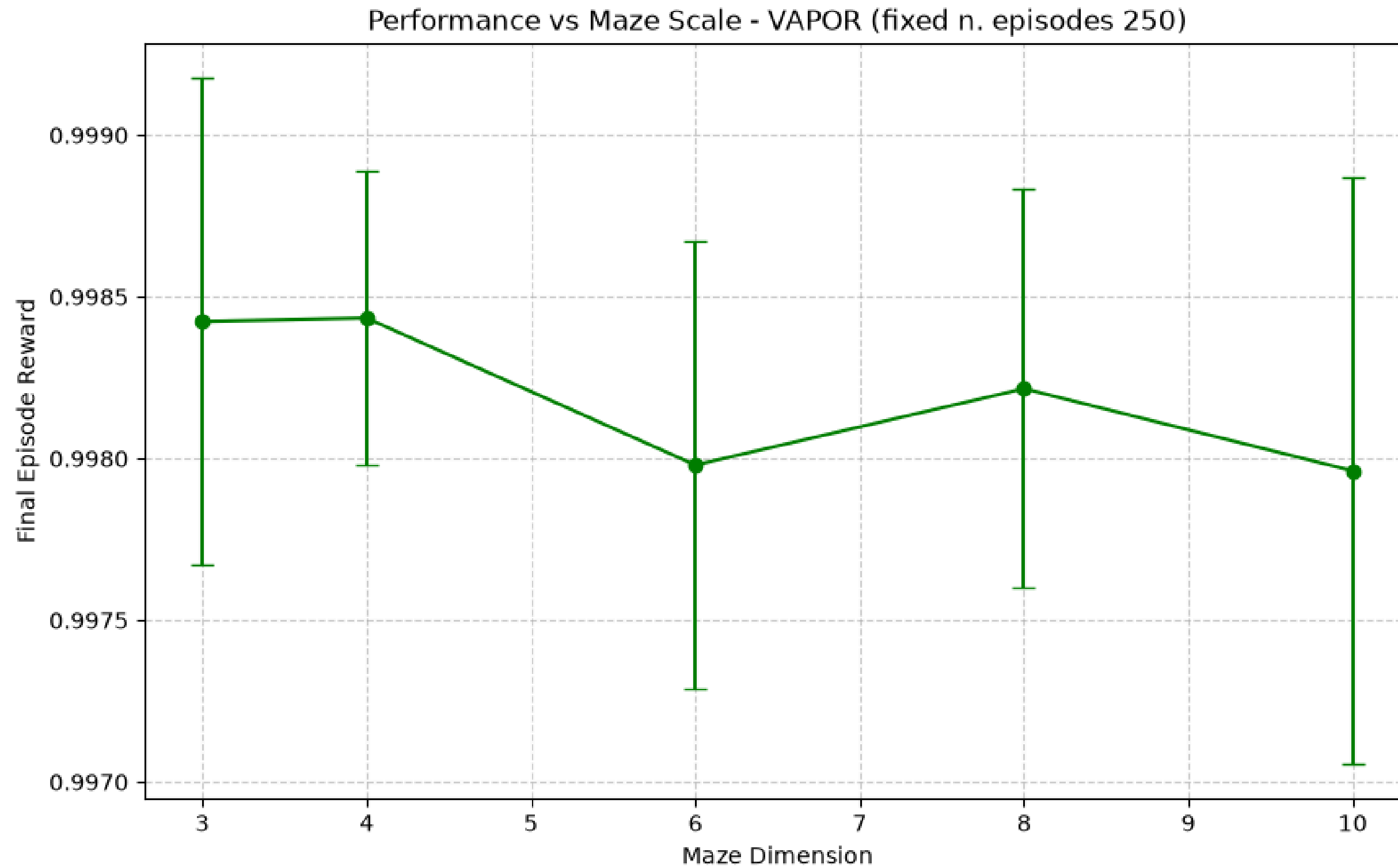


Results: Soft Q-learning

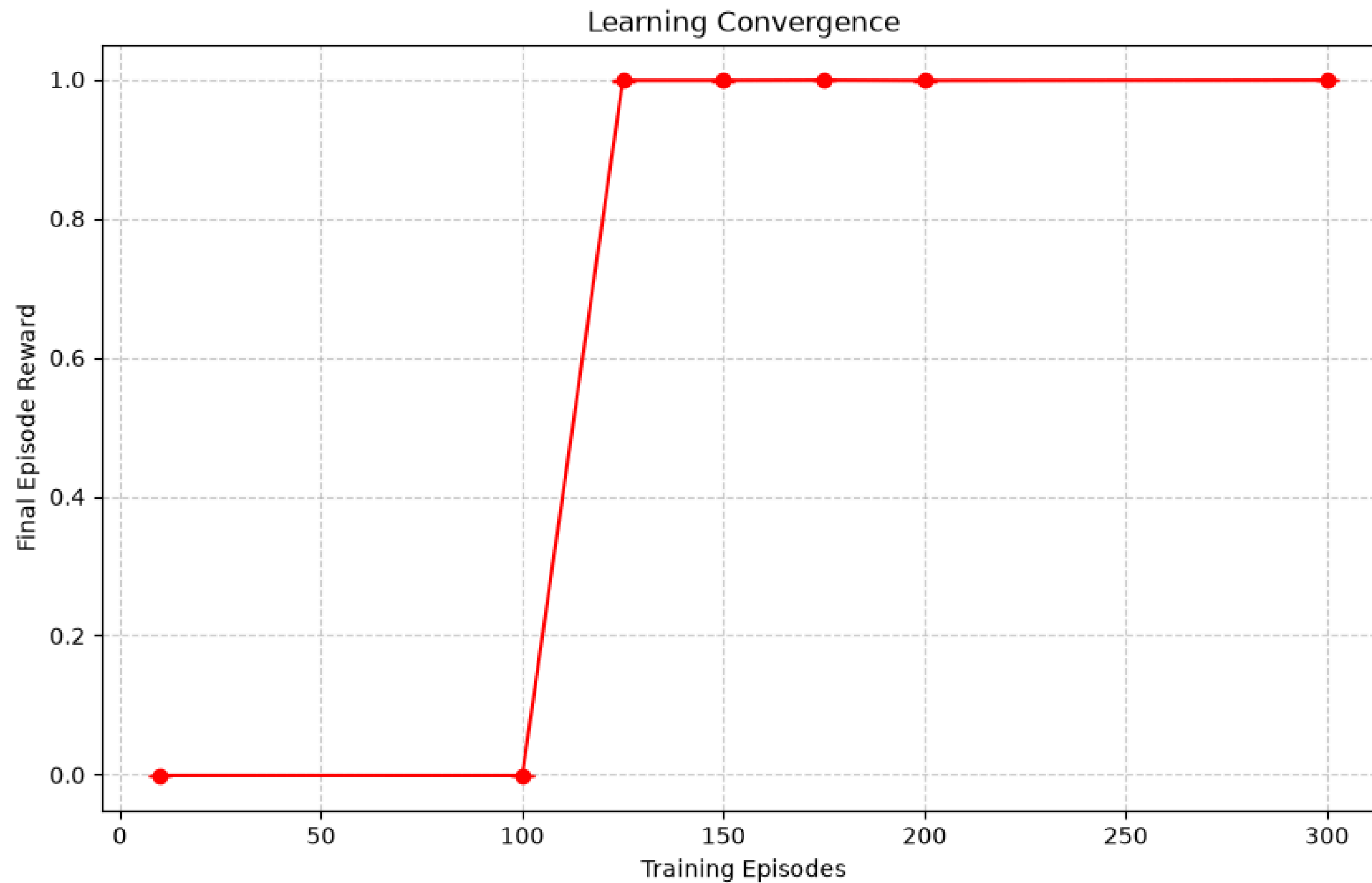


VAPOR

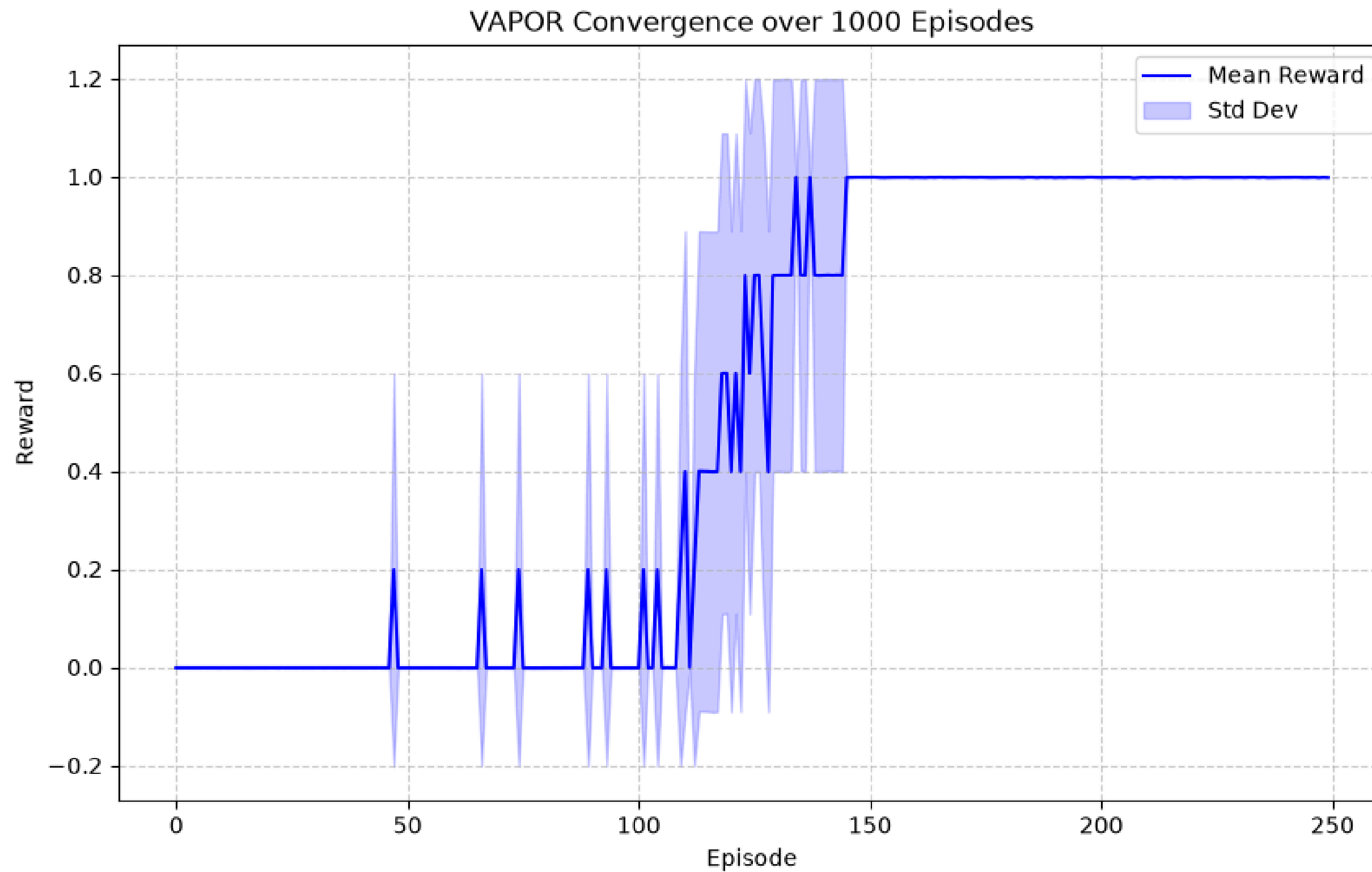
Results: VAPOR



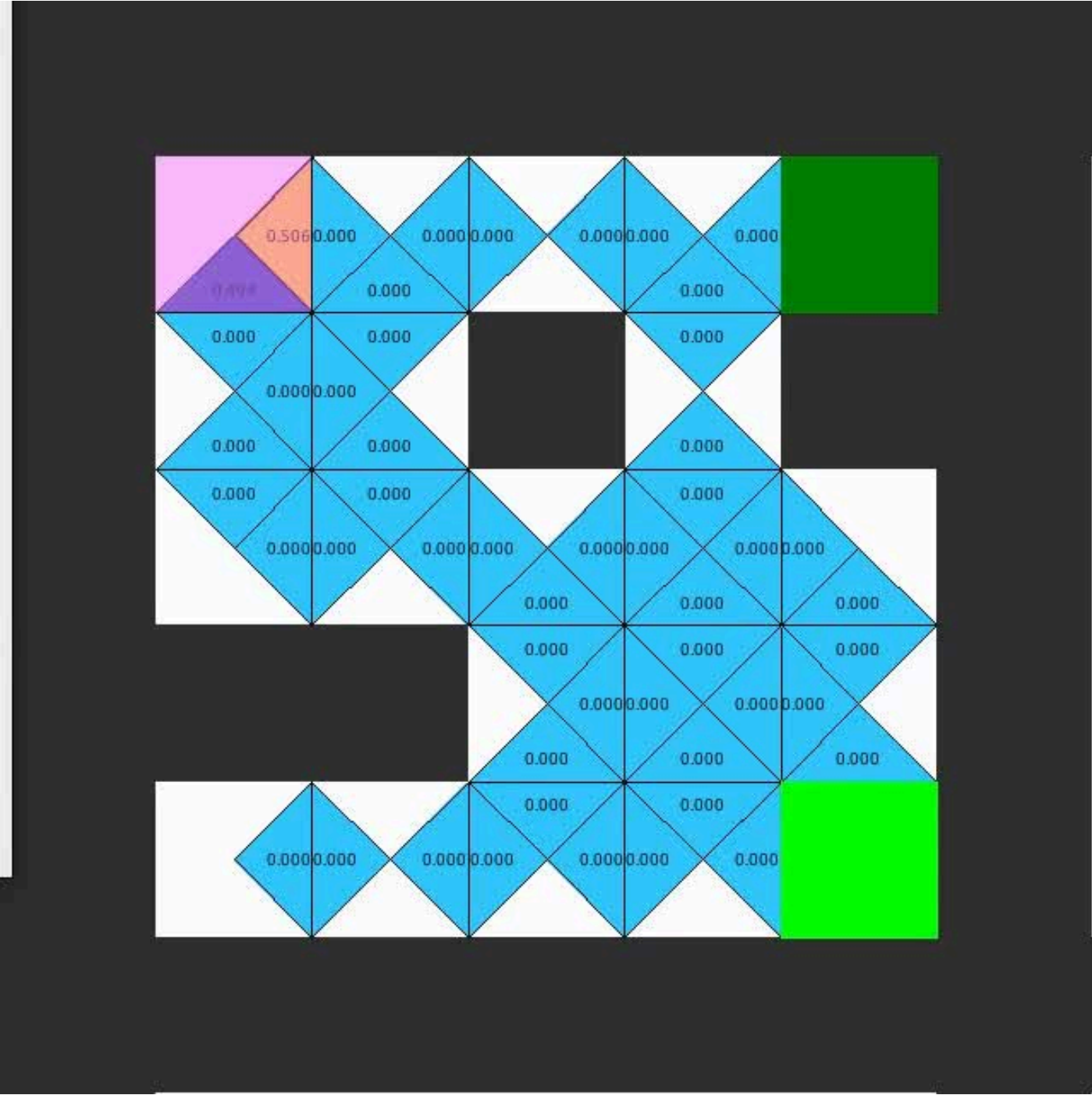
Results: VAPOR



Results: VAPOR



Results: VAPOR



Conclusions

On one hand...

- We can clearly notice that **VAPOR**, is able to arrive to the solution in way less episodes than the other two algorithms tested. This we believe is due to the better use of the epistemic uncertainty which is letting us explore in a more accurate way.
- Testing on simple bandit-like problems yields that results are in line with what was seen in the gridworlds simulations

On the other hand...

- Implementing efficiently the convex optimisation step proved to be more challenging than initially planned...
- It is still a tabular method, so it will **NOT** does not scale well. (See VAPOR-lite)



Any questions?